Software Test Plan (STP)

Blood Bank Management System (BBMS)

Field	Details
Project	Blood Bank Management System (BBMS)
Team	Team 9
Team Members	Avaneesh Aroor, Avrit Sharma, Ayush Kumar, Ayushman Dey
Version	1.0
Date	17-09-2026
Status	Draft
Reference SRS	Team 9 Blood Bank Management System SRS, v1.0

1. Introduction

Purpose

This document lays out how Team 9 is going to test the Blood Bank Management System before it goes into acceptance. It walks through what gets tested, how deep the testing goes at each stage, who is responsible for what, and what has to be true before we can call the system ready for sign-off. It is built directly on top of the requirements captured in the project SRS, so every test activity here traces back to something that document actually asks for.

Scope

Testing covers the full application as scoped in the SRS: user registration and login, donor and donation management, blood inventory tracking, blood requests and fulfillment, notifications and reporting, and the administration and audit functions. It also covers the non-functional and security requirements tied to those modules, things like response time, availability, password handling, and session expiry.

A few things sit outside what we're testing. We are not verifying laboratory equipment internals, we are not testing any external hospital system beyond the interfaces the SRS defines, and where the notification service is backed by a third-party email or SMS provider, we are only testing that our system triggers and logs the notification correctly, not the provider's own delivery pipeline.

References

- Team 9 Blood Bank Management System SRS, v1.0
- Requirements Traceability Matrix in Section 8 of the SRS
- UML Use-Case Diagrams (Figures 1 and 2 of the SRS)

Definitions

BBMS (Blood Bank Management System), UI (User Interface), API (Application Programming Interface), DBMS (Database Management System), CRUD (Create, Read, Update, Delete), RBAC (Role-Based Access Control), HTTPS (Hypertext Transfer Protocol Secure), JWT (JSON Web Token), SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), TC (Test Case).

2. Test Items

- User Authentication and Profile module
- Donor and Donation Management module
- Blood Inventory Management module
- Blood Request and Fulfillment module
- Notifications and Reports module
- Administration and Audit module
- Security and access-control layer (spans all modules)

3. Features to be Tested

Every feature below is mapped to its requirement ID in the SRS. The complete requirement-to-test-case mapping, including the ones not called out individually here, lives in the RTM in Section 13.

3.1 Authentication and Profiles

- BBMS-F-001: New user registration with unique account creation
- BBMS-F-002: Login and secure session establishment
- BBMS-F-003: Role-based access enforcement across Donor, Recipient, Staff, and Administrator
- BBMS-F-004: Viewing and updating permitted profile fields

3.2 Donor and Donation Management

- BBMS-F-010: Scheduling a donation appointment
- BBMS-F-011: Recording a completed donation
- BBMS-F-012: Rejecting duplicate active appointments
- BBMS-F-013: Donor eligibility checks (age, weight, interval since last donation)

3.3 Blood Inventory Management

- BBMS-F-020: Creating an inventory record for a verified blood unit
- BBMS-F-021: Storing and validating all mandatory unit fields
- BBMS-F-022: Stock updates on reservation, issue, expiry, discard, and return
- BBMS-F-023: Searching available blood by group and component
- BBMS-F-024: Automatic expiry marking via the scheduled daily job

3.4 Blood Request and Fulfillment

- BBMS-F-030: Submitting a blood request
- BBMS-F-031: Approving, rejecting, or holding a request
- BBMS-F-032: Reserving and issuing units against an approved request
- BBMS-F-033: Requesters viewing their own request status and history
- BBMS-F-034: ABO and Rh compatibility enforcement (safety-critical, tested separately in Section 5.1)

3.5 Notifications and Reports

- BBMS-F-040: Notifications on request status changes
- BBMS-F-041: Low-stock alerts at a configurable threshold
- BBMS-F-042: Reports for donations, inventory, requests, and issued units

3.6 Administration and Audit

- BBMS-F-050: Admin management of staff accounts and roles
- BBMS-F-051: Audit trail for logins and critical changes

3.7 Non-functional and Security

- BBMS-NF-001 to NF-006: Response time, availability, password hashing, audit retention, accessibility, backup and recovery
- BBMS-SR-001 to SR-005: TLS enforcement, server-side authorization, input sanitization, security event logging, session timeout

4. Features Not to be Tested

- Physical blood testing and laboratory equipment internals

- External hospital systems beyond the interfaces defined in the SRS
- Delivery internals of any third-party email or SMS provider (only the trigger and logging on our side are tested)
- Underlying reliability of the database engine itself, that is assumed to be handled by PostgreSQL or MySQL

5. Test Approach / Strategy

Levels

- Unit tests at the module level (Django/Node components in isolation)
- Integration tests between modules and between the application and the database
- System tests covering end-to-end BBMS workflows
- Acceptance tests (UAT) with representative donor, recipient, staff, and admin users

Types

- Functional testing of all core features listed in Section 3
- Regression testing after every significant fix or feature addition
- Performance testing for response time and load, per BBMS-NF-001
- Usability and accessibility testing of the web UI, per BBMS-NF-005

Entry criteria: a stable build is available, test data covering all donor, recipient, and blood-unit scenarios is loaded, and the test environment is provisioned.

Exit criteria: every planned test case has been executed, there are zero open critical defects, and every high-priority functional and security requirement has been verified against the SRS.

5.1 Blood Type Compatibility Testing

BBMS-F-034 is safety-critical, so it gets its own dedicated test pass rather than being folded into general functional testing. We build a full compatibility matrix covering every ABO and Rh donor-recipient combination, including the edge cases that are easy to get wrong: O- as the universal donor, AB+ as the universal recipient, and the Rh- to Rh-only rule. Every incompatible pairing in the matrix has to be rejected by the system, and every compatible pairing has to be accepted, before this area is signed off.

5.2 Scheduled Job Testing

The daily expiry job (BBMS-F-024) runs unattended, so we test it with manipulated system or unit expiry dates rather than waiting for real time to pass. This includes units expiring exactly at the job's run boundary, units that expired days earlier and should already be excluded, and a check for what happens if the job itself fails partway through a run.

5.3 Role-Based Access and Permission Matrix Testing

For BBMS-F-003 and BBMS-SR-002, we build a permission matrix crossing all four roles against every protected operation in the system. Each cell in the matrix gets both a positive test, confirming the right role can perform the action, and a negative test, confirming every other role is denied.

5.4 Security Validation

- TLS 1.2+ verification on all client-server communication
- Confirming passwords are hashed and never stored or logged in plaintext

- Input sanitization and fuzzing on request forms, donor forms, and login fields to check for injection
- Verifying security-relevant events (login failures, role changes, inventory issues, approvals) land in the audit log
- Session timeout behavior after configured inactivity

6. Test Environment

Hardware

Development laptops running the test suite locally, a cloud VM or shared server for the integration environment, and an optional barcode/QR scanner for blood-unit identification testing.

Software

BBMS application (Django or Node.js backend), PostgreSQL or MySQL sandbox database, and a standard web browser client.

Tools

- Selenium for UI automation
- Postman for API-level testing
- JMeter for performance and load testing
- Pytest or Jest for unit tests, depending on the backend chosen
- GitHub Issues for defect tracking

Test Data

- Dummy donor, recipient, staff, and admin accounts covering all four roles
- Sample blood units across all eight ABO/Rh groups, including some already expired and some close to expiry
- A set of pre-built blood requests at various stages (pending, approved, rejected, fulfilled)

7. Test Schedule

Milestone	Date
Test case design	28-Sep-2026
Environment setup	02-Oct-2026
Test execution start	03-Oct-2026
Test execution end	13-Oct-2026
UAT	14-Oct-2026 to 16-Oct-2026

8. Test Deliverables

- Test Plan (this document)
- Test cases, manual and automated
- Test scripts
- Test data sets

- Test execution logs
- Defect reports
- Test summary report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Avaneesh Aroor	Prepares the test plan and coordinates execution across the team
Test Engineer	Ayush Kumar	Designs and executes test cases, logs defects
Dev Lead	Avrit Sharma	Supports defect triage and fixes
Product Owner	Ayushman Dey	Approves test results and signs off on release readiness

10. Risks and Mitigation

Risk	Mitigation
Blood-type compatibility bugs slip through and surface late	Run the compatibility matrix in Section 5.1 early, before broader system testing starts
The scheduled expiry job behaves differently in test than in production	Expose a manual trigger for the job in the test environment instead of waiting on real time
Test data doesn't cover all blood groups and edge cases	Build the synthetic dataset described in Section 6 before execution begins
Team availability is tight across four members with overlapping coursework	Assign module ownership early so testing isn't blocked on any one person

11. Assumptions and Dependencies

- The database sandbox stays stable and available for the full test window
- Test accounts for all four roles are provisioned before execution starts
- The notification service is available in a sandbox or mock mode during testing

12. Suspension and Resumption Criteria

Testing is suspended if the environment is unavailable for more than four hours, or if the build is unstable enough to block more than 30% of planned test cases.

Testing resumes once blocking defects are resolved and the environment is confirmed stable again.

13. Test Case Management and Traceability

The SRS already includes a full Requirements Traceability Matrix mapping every BBMS-F, BBMS-NF, and BBMS-SR requirement to a test case ID, with all statuses currently marked N (not run). That matrix carries forward as the backbone of this test plan; as execution proceeds, the status column updates to P (pass) or a defect reference. A few representative entries:

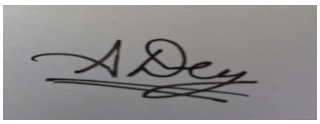
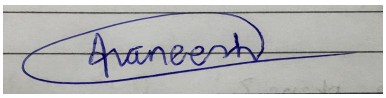
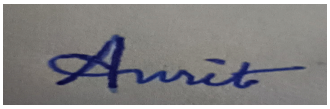
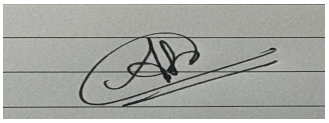
Requirement	Description	Test Case
BBMS-F-001	User registration	TC-Auth-01
BBMS-F-013	Donor eligibility checks	TC-Don-04
BBMS-F-024	Automatic expiry marking	TC-Inv-05
BBMS-F-034	ABO/Rh compatibility enforcement	TC-Req-05
BBMS-NF-001	Response time under load	TC-Perf-01
BBMS-SR-003	Input sanitization	TC-Sec-03

14. Test Metrics and Reporting

- Percentage of test cases executed
- Percentage passed versus failed
- Defect density and defect aging
- Requirement coverage against the RTM

Reports: a running daily execution status during the test window, and a final test summary report once UAT closes.

15. Approvals

Role	Name	Signature
Product Owner	Ayushman Dey	
QA Lead	Avaneesh Aroor	
Dev Lead	Avrit Sharma	
Test Engineer	Ayush Kumar	
Course Coordinator	Prof. Asha Kumari	ASHA KUMARI